

Data mining & machine learning

5. Explainability

Francesco Pistolessi

University of Pisa

`francesco.pistolessi@unipi.it`

M.Sc. in Artificial Intelligence and Data Engineering



UNIVERSITÀ DI PISA

Why do we need explanations?

A model prediction is often a number: class, probability, score. An explanation connects that number to the input features.

Main question

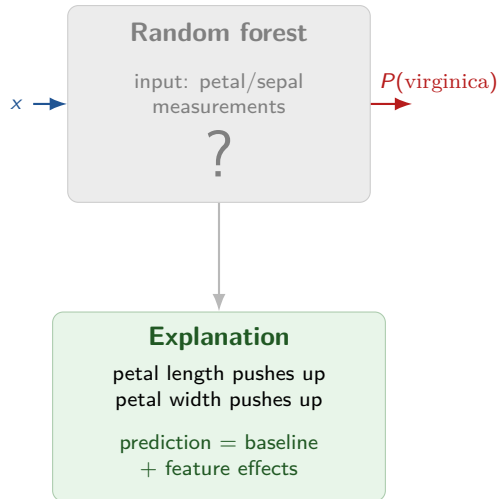
For a flower x , the model predicts *virginica* with probability 0.991. Which feature values made this probability high?

Two types of answers:

- **Global:** which features matter for the model in general?
- **Local:** why did the model give this prediction to this sample?

Methods in this lecture:

- Feature importance: simple global ranking.
- SHAP: additive feature contributions.
- LIME: simple local model that imitates the black box near one sample.



Running example: Iris classification

We use **one dataset throughout the practical part**: Iris.

Each row is one flower. The four features are:

- sepal length, sepal width;
- petal length, petal width.

The target has three classes:

Class index	Class name
0	setosa
1	versicolor
2	virginica

We will explain the probability of class 2:

$$f_2(x) = P(Y = \text{virginica} \mid x).$$

```
import numpy as np
import pandas as pd
import shap
from sklearn.datasets import load_iris
from sklearn.model_selection import
    train_test_split
from sklearn.ensemble import
    RandomForestClassifier
from sklearn.metrics import
    accuracy_score

iris = load_iris(as_frame=True)
X = iris.data.copy()
y = iris.target.copy()
class_names = iris.target_names.tolist()

class_to_explain = 2
class_label = class_names[
    class_to_explain]
# class_label is 'virginica'
```

Notebook setup: same variables in every snippet

```
X_train, X_test, y_train, y_test =  
    train_test_split(  
        X, y, test_size=0.30,  
        random_state=42, stratify=y)  
  
rf = RandomForestClassifier(  
    n_estimators=300, max_depth=4,  
    random_state=42)  
rf.fit(X_train, y_train)  
  
print(accuracy_score(y_test, rf.predict(X_test  
    )))  
# 0.911 in this run  
  
sample_position = 0  
sample_df = X_test.iloc[[sample_position]]  
pred_prob = rf.predict_proba(sample_df)[0,  
    class_to_explain]  
print(class_label, pred_prob)  
# virginica 0.991...
```

Meaning of the variables

X, y	all data and labels
X_train, X_test	features split in train/test
rf	trained random forest
class_to_explain	class index, here 2
class_label	class name, here virginica
sample_position	row position in X_test
sample_df	one test row as a DataFrame

What does iloc mean?

iloc selects rows by **integer position**.
X_test.iloc[0] returns the first test row as a Series.
X_test.iloc[[0]] keeps it as a one-row DataFrame, which is what predict_proba expects.

Global and local explanations

Global explanation

Answers a question about the model as a whole.

“Which features usually matter?”

Example: petal length is important for many Iris predictions.

Local explanation

Answers a question about one selected sample x^* .

“Why this prediction for this flower?”

Example: this flower has a long petal and wide petal, so $P(\text{virginica})$ becomes high.

Method	Global	Local
Tree feature importance	yes	no
Permutation importance	yes	no
SHAP bar / beeswarm	yes	no
SHAP waterfall / force	no	yes
LIME	no	yes

A local explainer

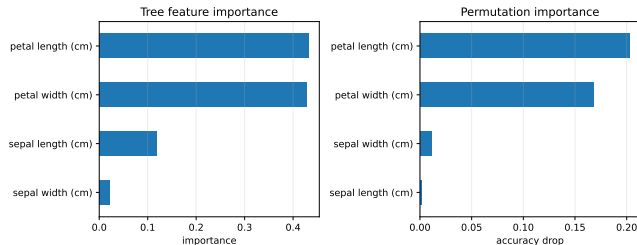
A **local explainer** explains one input x^* . Its explanation should not be read as a statement about the entire model.

Feature importance: a global first check

Feature importance ranks features by how much the model relies on them.

Two versions:

- **Tree importance:** computed from the splits used by the forest.
- **Permutation importance:** damages one feature column and measures the performance drop.



What it does not tell us

It does not explain one specific flower and it does not say whether a feature pushes the probability up or down.

```
imp = pd.Series(rf.feature_importances_,
                index=X_train.columns)

perm = permutation_importance(
    rf, X_test, y_test,
    n_repeats=40, random_state=42)
```

What does it mean to permute a feature?

To **permute** a feature means to shuffle the values in one column among the test samples.

- 1 Choose one column, e.g. petal length (cm).
- 2 Copy `X_test`; keep the trained model unchanged.
- 3 Randomly rearrange only that column.
- 4 Predict again on the damaged test set.
- 5 If accuracy drops a lot, the model needed that column.

Intuition

After shuffling, petal length values no longer belong to the correct flowers. The column still has realistic numbers, but the relation with the other features and with the label has been broken.

Original test rows

	petal length	petal width
row 0	6.3	1.8
row 1	4.7	1.4
row 2	5.1	1.5
row 3	4.7	1.6
row 4	4.9	1.8

After permuting petal length

	petal length	petal width
row 0	5.1	1.8
row 1	6.3	1.4
row 2	4.9	1.5
row 3	4.7	1.6
row 4	4.7	1.8

```
feature = 'petal length (cm)'  
rng = np.random.default_rng(42)  
X_perm = X_test.copy()  
X_perm[feature] = rng.permutation(  
    X_test[feature].to_numpy())  
  
original_acc = accuracy_score(  
    y_test, rf.predict(X_test))  
permuted_acc = accuracy_score(  
    y_test, rf.predict(X_perm))  
importance = original_acc - permuted_acc
```

SHAP: the main idea in one sentence

SHAP explains a prediction as:

$$\text{prediction} = \text{baseline} + \text{feature contributions}.$$

For class $k = 2$ (virginica) and one flower x^* :

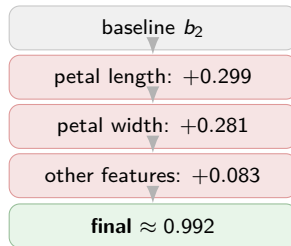
$$f_2(x^*) \approx b_2 + \sum_{j=1}^p \phi_{j,2}(x^*).$$

Meaning of the symbols

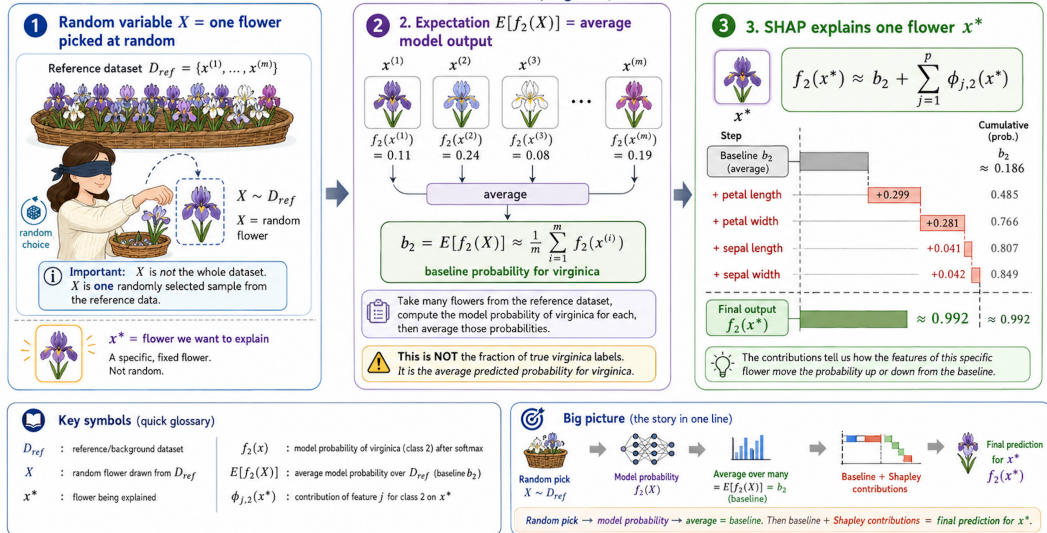
x^*	flower whose classification we want to explain
$f_2(x^*)$	model probability of <i>virginica</i> for that flower
$b_2 = E[f_2(X)]$	average/reference model output for <i>virginica</i>
$\phi_{j,2}(x^*)$	contribution of feature j for class 2

How to (intuitively) read the equation

Start from the model's usual probability for *virginica*. Then, add how much this flower's petal length, petal width, sepal length, and sepal width move that probability up or down.



The big picture



What is $E[f_2(X)] = 0.328$ for a classifier?

Here $f_2(x)$ is not a class label. It is a **probability**:

$$f_2(x) = P(Y = \text{virginica} \mid x).$$

X means “a flower drawn from the background data”, here the training set. Therefore

$$E[f_2(X)]$$

means: average the model's virginica probability over background flowers.

Interpretation of 0.328

Before looking at the specific test flower, the model's reference probability for virginica is about **32.8%**. It is the starting point for SHAP, not the final prediction for every flower.

```
explainer = shap.TreeExplainer(
    rf, data=X_train,
    feature_perturbation='interventional',
    model_output='probability')

shap_values = explainer(
    X_test, check_additivity=False)

sv_class = shap_values[:, :,
    class_to_explain]
base_value = sv_class.base_values[
    sample_position]
print(base_value)
# 0.328...
```

Why close to one third?

Iris has three balanced classes. If we draw a random flower before seeing its measurements, virginica is roughly one class out of three. The model's average predicted probability is therefore close to $1/3$.

SHAP additivity for one flower

For test row 0, the flower has long petals:

Feature	Value
sepal length	7.3
sepal width	2.9
petal length	6.3
petal width	1.8

SHAP decomposes the virginica probability as:

$$\begin{aligned}P(\text{virginica} \mid \mathbf{x}^*) &\approx 0.328 + 0.664 \\ &\approx 0.992.\end{aligned}$$

Important

The base value is average behaviour. The SHAP values explain why this specific flower moves far above that average.

```
phi = sv_class.values[sample_position]
base = sv_class.base_values[sample_position]
final_from_shap = base + phi.sum()

final_from_model = rf.predict_proba(
    sample_df)[0, class_to_explain]

print(base) # 0.328...
print(phi.sum()) # 0.664...
print(final_from_shap) # 0.992...
print(final_from_model) # 0.991...
```

Feature contributions

Feature	SHAP value
petal length	+0.299
petal width	+0.281
sepal length	+0.075
sepal width	+0.008

Probability and log-odds: do not mix the scales

A classifier can output probabilities or scores.
SHAP values must be interpreted on the same scale as the output being explained.

Probability scale

In these slides we explain

$$f_2(x) = P(Y = \text{virginica} \mid x).$$

So $\phi = +0.10$ means “add 10 percentage points to the virginica probability relative to the base value”.

Practical rule

If the plot is on the probability scale, read values as probabilities. If the plot is on the log-odds scale, read values as log-odds. Never mix them.

What are odds and log-odds?

For a probability p :

$$\text{odds} = \frac{p}{1-p}, \quad \text{log-odds} = \log \frac{p}{1-p}.$$

Example: if $p = 0.80$, then odds are 4 to 1 and log-odds are $\log(4)$.

Why log-odds appear in some tools

Probabilities are bounded between 0 and 1. Log-odds can take any real value, so additive decompositions are often easier on that scale. Here we avoid this ambiguity by using `model.output='probability'`.

What is a SHAP explainer?

An **explainer** is the object that *knows how to compute explanations for a model*.

It needs two inputs:

- the trained model `rf`;
- background data `X_train`, used as the reference population.

What it returns

`shap_values` contains:

- `.base_values`: the baseline b_k ;
- `.values`: the feature contributions $\phi_{j,k}$;
- `.data`: the feature values displayed in plots.

```
explainer = shap.TreeExplainer(
    rf, data=X_train,
    feature_perturbation='interventional',
    model_output='probability')

shap_values = explainer(
    X_test, check_additivity=False)

# shape: samples x features x classes
print(shap_values.values.shape)
# (45, 4, 3)

# Take only class 2: virginica
sv_class = shap_values[:, :,
    class_to_explain]
```

Why select one class?

Multiclass classifiers have one output per class. We must choose which output we explain. Here:
 $f_2(x) = P(\text{virginica} \mid x)$.

SHAP bar plot: global importance

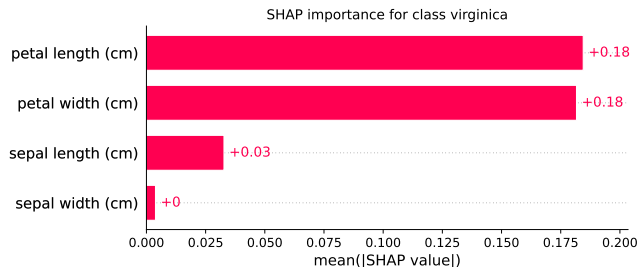
The bar plot averages absolute SHAP values over many samples:

importance of feature $j = E [|\phi_{j,2}(X)|]$.

How to read it:

- longer bar = larger average effect on $P(\text{virginica})$;
- it is global: it summarizes many flowers;
- it has no sign: positive and negative effects are both counted by magnitude.

```
shap.plots.bar(  
    sv_class, max_display=4)
```

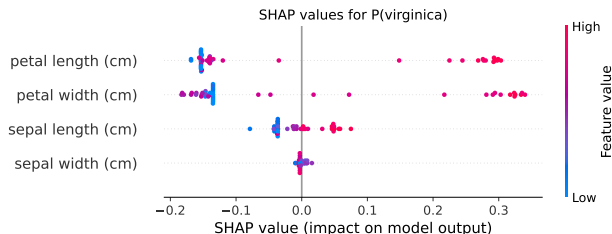


SHAP beeswarm: importance, sign, and feature values

The beeswarm shows one dot per flower and per feature.

How to read it:

- x-axis = SHAP value for $P(\text{virginica})$;
- right side = feature pushes probability up;
- left side = feature pushes probability down;
- colour = original feature value: high or low.



Intuition

If high petal width values are mostly on the right, large petal width makes the model more confident in virginica.

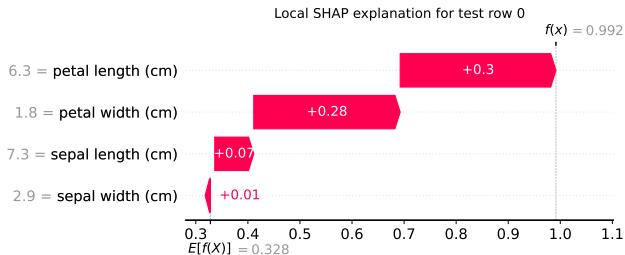
```
shap.plots.beeswarm(  
    sv_class, max_display=4)
```

SHAP waterfall: local explanation for one flower

The waterfall explains a selected flower x^* .

How to read it:

- 1 Start at $E[f_2(X)] \approx 0.328$.
- 2 Add each feature contribution.
- 3 Red bars increase $P(\text{virginica})$.
- 4 Blue bars decrease it.
- 5 End at the model prediction, about 0.991.



```
shap.plots.waterfall(  
    sv_class[sample_position],  
    max_display=4)
```


SHAP force and scatter plots

Force plot: same information as the waterfall, compressed.

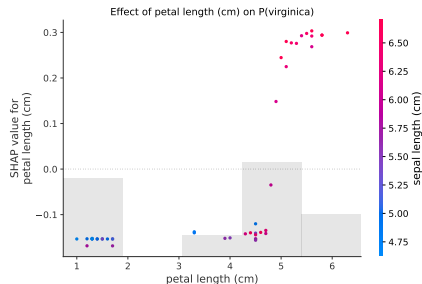
Red contributions push the probability up; blue contributions push it down.

```
shap.force_plot(  
    base_value=sv_class.base_values[  
        sample_position],  
    shap_values=sv_class.values[  
        sample_position],  
    features=X_test.iloc[sample_position],  
    feature_names=X_test.columns,  
    matplotlib=True)
```



Scatter plot: effect of a feature over samples. x-axis is the feature value; y-axis is its SHAP contribution.

```
mean_abs_shap = np.abs(sv_class.values).mean(  
    axis=0)  
top_feature = X_test.columns[np.argmax(  
    mean_abs_shap)]  
shap.plots.scatter(sv_class[:, top_feature],  
    color=sv_class)
```



Multiclass SHAP: one explanation per class

Iris has three class outputs:

$$f_0(x) = P(\text{setosa} \mid x),$$

$$f_1(x) = P(\text{versicolor} \mid x),$$

$$f_2(x) = P(\text{virginica} \mid x).$$

For every class k :

$$f_k(x^*) \approx b_k + \sum_j \phi_{j,k}(x^*).$$

Do not mix classes

A feature can increase $P(\text{virginica})$ and decrease $P(\text{versicolor})$ for the same flower. Always state which class you are explaining.

```
# All classes are in shap_values:  
# axis 0: samples  
# axis 1: features  
# axis 2: classes  
print(shap_values.values.shape)  
# (45, 4, 3)  
  
# class 0: setosa  
sv_setosa = shap_values[:, :, 0]  
  
# class 1: versicolor  
sv_versicolor = shap_values[:, :, 1]  
  
# class 2: virginica  
sv_virginica = shap_values[:, :, 2]
```

In these slides

All SHAP plots after setup use `sv_class`, which means `sv_virginica`.

What changes for regression? Only the units

For regression there is one scalar output, not one output per class. If the model predicts a target $\hat{y}$:

$$f(x) = \hat{y}(x), \quad \hat{y}(x^*) \approx E[\hat{y}(X)] + \sum_j \phi_j(x^*).$$

Interpretation

If the target is a price, SHAP values are in price units. If the target is a temperature, SHAP values are in temperature units. The baseline is the average model prediction over background examples.

Difference from classification

Regression: one equation, one output scale.

Classification: choose the class k first; explain $f_k(x)$.

Same intuition

Start from a typical model output. Add the feature contributions. Arrive at the prediction for the selected sample.

LIME means **Local Interpretable Model-agnostic Explanations**.

It explains one sample x^* by building a simple model that imitates the random forest only near x^* .

- 1 Choose the flower x^* to explain.
- 2 Generate many synthetic neighbours z_i near x^* .
- 3 Ask the random forest for $f_2(z_i) = P(\text{virginica} \mid z_i)$.
- 4 Give higher weights to neighbours closer to x^* .
- 5 Fit a simple weighted linear model g_2 to imitate f_2 locally.

What is the surrogate model?

The surrogate g_2 is a simple model used only for explanation:

$$g_2(z) = \beta_0 + \sum_j \beta_j z_j.$$

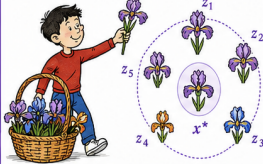
The coefficients β_j become the local explanation.

Key intuition

LIME does not try to understand the whole forest. It asks: "Around this flower, which linear rule best mimics the forest?"

The big picture

1. Choose the flower x^* and create synthetic neighbours



Flower x^* (target)

sepal length	6.4 cm
sepal width	2.8 cm
petal length	5.6 cm
petal width	2.2 cm

Each z_i is one synthetic neighbour flower (one full input sample).

Classifier output:

$$P(\text{virginica} | x^*) = 0.86$$



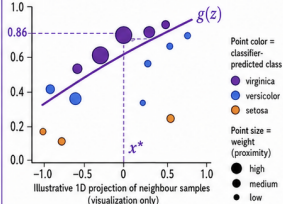
2. Query the classifier and weight by proximity

Neighbour flower sample	Classifier output $P(\text{virginica} z_i)$	Proximity / weight	
z_1	→ 0.92	distance	weight (proximity)
z_2	→ 0.81	z_1	0.10 0.95
z_3	→ 0.74	z_2	0.18 0.84
z_4	→ 0.38	z_3	0.30 0.68
z_5	→ 0.12	z_4	0.55 0.29
		z_5	0.82 0.09

- Closer neighbours matter more in the local fit.
- LIME uses the classifier probabilities, not the true labels.

3. Fit a local surrogate regression

Predicted probability $P(\text{virginica})$



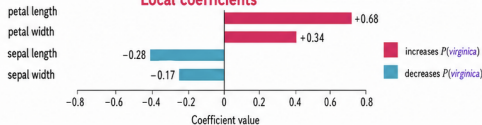
The x -axis is not one original feature. It only arranges neighbour flowers for visualization.

$$g(z) = \beta_0 + \beta_1 (\text{petal length}) + \beta_2 (\text{petal width}) + \beta_3 (\text{sepal length}) + \beta_4 (\text{sepal width})$$

Here z is one neighbour flower sample; the terms in parentheses are its feature values.

The surrogate is not the classifier: it is a simple regression that imitates it only near x^* .

Local coefficients



Core idea

LIME does not try to understand the whole classifier. It creates neighbour flowers around x^* , observes how the classifier behaves there, and fits a simple local rule that explains that one prediction.

LIME neighbours: what are the synthetic points?

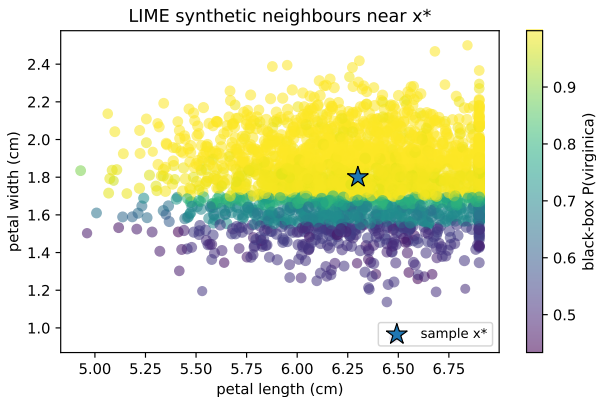
Synthetic neighbours are artificial flowers created by slightly changing the feature values of x^* .

Why generate them?

A single point is not enough to fit a local line. LIME creates nearby points, queries the black-box model on them, and observes how the model output changes around x^* .

They are not new labelled data

The synthetic neighbours do not need true Iris labels. Their target values are **the random forest probabilities**. LIME explains the model, not the data-generating process.



LIME in code I: perturb the sample and query the model

```
sample_position = 0
sample_df = X_test.iloc[[sample_position]]
x0 = X_test.iloc[sample_position]

rng = np.random.default_rng(42)
feature_mean = X_train.mean()
feature_std = X_train.std().replace(0, 1)

n_perturb = 2500
Z = pd.DataFrame(
    rng.normal(loc=x0.values,
              scale=0.25 * feature_std.values,
              size=(n_perturb, X_train.shape[1])),
    columns=X_train.columns)
Z = Z.clip(lower=X_train.min(),
          upper=X_train.max(), axis=1)

black_box_output = rf.predict_proba(Z)[: ,
                                     class_to_explain]
```

Meaning of the code

- `x0`: the flower to explain.
- `Z`: synthetic neighbours around `x0`.
- `black_box_output`: random forest probabilities on `Z`.
- `feature_std`: makes noise proportional to each feature scale.

Why use clip?

Random perturbations can create unrealistic values. Clipping keeps synthetic features within the range observed in the training data.

LIME in code II: fit the local surrogate

```
from sklearn.linear_model import Ridge
from sklearn.metrics import r2_score

Z_scaled = (Z - feature_mean) / feature_std
x0_scaled = (x0 - feature_mean) / feature_std

distances = np.sqrt(
    ((Z_scaled - x0_scaled) ** 2).sum(axis=1))

kernel_width = 0.75 * np.sqrt(X_train.shape[1])
neighbor_weights = np.exp(
    -(distances ** 2) / (kernel_width ** 2))

local_model = Ridge(alpha=0.1)
local_model.fit(Z_scaled, black_box_output,
               sample_weight=neighbor_weights)

lime_weights = pd.Series(local_model.coef_,
                        index=X_train.columns)
local_pred = local_model.predict(Z_scaled)
print(r2_score(black_box_output, local_pred,
               sample_weight=neighbor_weights))
# 0.66 in this run
```

What is being fitted?

The surrogate tries to match the random forest probabilities:

$$\min_{g_2} \sum_i \pi_i (f_2(z_i) - g_2(z_i))^2.$$

Here π_i is large when z_i is close to x^* .

Does LIME count neighbours classified correctly?

No. In this implementation, LIME fits the surrogate to the random forest's **probabilities**. A local fidelity score such as weighted R^2 tells us how well the surrogate imitates the forest near x^* .

LIME plot: local coefficients

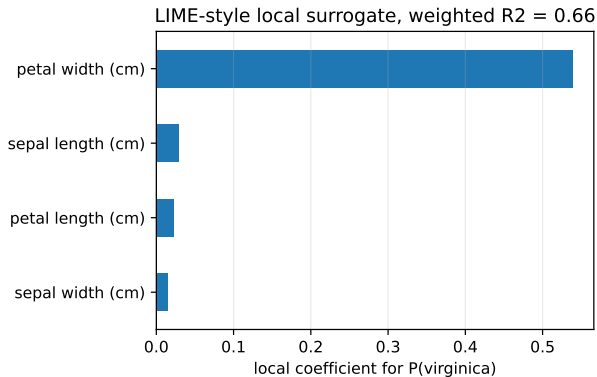
The bars are the coefficients of the local surrogate g_2 .

How to read them:

- positive coefficient: locally increases $P(\text{virginica})$;
- negative coefficient: locally decreases $P(\text{virginica})$;
- larger magnitude: stronger local effect near x^* .

Important

These coefficients are local. They describe the surrogate near this flower, not a global law for all Iris flowers.



SHAP vs. LIME: what should you remember?

	SHAP	LIME
Core idea	baseline + feature contributions	local surrogate model
Output	$\phi_{j,k}$ values	surrogate coefficients β_j
Scope	local and global plots	local only
Classifiers	choose class k first	choose class output f_k first
Scale	probability, log-odds, or target units	usually probability or score
Main check	additivity	local fidelity

Recommended workflow:

- feature importance for a quick global view;
- SHAP beeswarm for direction and global behaviour;
- SHAP waterfall for one prediction;
- LIME as local cross-check.

Common mistakes to avoid

- 1. Unclear output.** Always state what $f_k(x)$ is. Here it is $P(\text{virginica} \mid x)$.
- 2. Mixing classes.** SHAP values for setosa, versicolor, and virginica are different explanations.
- 3. Mixing scales.** Probability SHAP values are not log-odds SHAP values.
- 4. Treating LIME as global.** LIME explains one point only.

Core concepts

- A classifier output must be defined before explanation.
- In these slides: $f_2(x) = P(\text{virginica} \mid x)$.
- $E[f_2(X)]$ is the reference probability over background flowers.
- SHAP adds feature contributions to this reference value.
- LIME fits a simple model near one sample.

Practical interpretation

For the selected flower:

$$0.328 + 0.299 + 0.281 + 0.075 + 0.008 \approx 0.991.$$

The long petal and wide petal explain why the model strongly predicts virginica.

state the output

state the class

state the scale

Questions?